

My contribution:

I owned the infrastructure that turns the four research modules into one tool you can run, and I built the Apache Commons Math 2.2 benchmark that we use to compare against published work. Also, wrote the “Solution” section of “Automatic Discovery of Metamorphic Relations for Mathematical functions”

I wrote the command-line entry point (`amr/cli.py`) and the per-function pipeline (`amr/run.py`). This is the glue that runs each function through reconnaissance, bound translation, the PSO search, deduplication, and mutant scoring, then writes one JSON result per function plus the `summary.csv`. The part I spent the most time on is the restart loop in `run_function`. A single PSO search returns one relation, and one relation rarely catches every fault, so I run eight restarts per function with a different seed each time, then tested it with 16 and 32 restarts, decided to stop on 16 restarts. The restarts alternate between two bound sets: the tight LLM-guided bounds find the canonical relation fast, and the wide default bounds wander further and pick up the odder relations that cover more mutants. I also handled the failure path, so a reconnaissance call that throws falls back to default bounds and the run keeps going instead of dying. I added the result and summary writers, the skip-if-exists behavior so reruns do not recompute finished functions, and the `-force` flag to override it.

I generated the per-function mutant specs under `data/mutants/` (about 78 per function) and ran the full experiment that produced the numbers in `results/`. The logs from those runs are in `logs/`. The headline per-function kill rates, from 32% on `tan` up to 79.5% on `log1p`, come from these runs

I added the second evaluation track that tests our relations against real library code rather than synthetic mutants. This is `amr/commons_math.py` (a port of the relevant Commons Math 2.2 functions), `amr/commons_mutants.py` (mutants over that port), and `amr/benchmark.py` (the reference loader and comparison builder). The runner and comparison scripts are `scripts/run_commons_benchmark.py` and `scripts/compare_benchmark.py`, and the output is `results/benchmark_comparison.md`. I tracked down the AutoMR and MRI reference numbers, stored them in `data/benchmark/automr_reference.json`, and documented the one inconsistency I could not resolve: the AutoMR paper's per-program seeded faults sum to 953 in Table V but the headline says 625. I left both numbers as published and noted it in the comparison file rather than quietly picking one. I wrote the tests for this work in `tests/test_benchmark.py` and `tests/test_commons.py`.

My commits:

44dc63b - infra: CLI, run loop, mutant data, first full result set

dd0250e - Commons Math 2.2 port, mutants, benchmark comparison, scripts, tests

PEER ASSESSMENT:

Hugo Tarczyński 10/10

Hugo built the search core: the PSO algorithm (`amr/pso.py`), the profile to bounds translation (`amr/bounds.py`), and the fitness function with filtering (`amr/mr.py`). This is the technical part of the project where a sloppy implementation would have quietly produced garbage relations. He got the details right. The fitness function does not just minimize residual, it carries three penalties that reject the degenerate answers a naive search would return: the all-zero relation, a transform that barely moves the input, and a near-zero scale. The validation step re-checks every candidate on a fresh seed, which is what stops us from reporting relations that only fit the training points. He also wrote the tests for his own modules

Ada Jacyna 10/10

Ada wrote the LLM reconnaissance module (`amr/recon.py`) and the shared interfaces that the project is built on (`amr/interfaces.py`). The interfaces were early and they were good, which is the main reason the rest of us could work in parallel without stepping on each other. The reconnaissance module constrains the model to a strict JSON schema, caches every answer to disk so reruns are free, and keeps the model strictly advisory so a wrong hint never breaks correctness. Ada also tested it properly, including mocking the model call so the suite runs without Ollama installed. Clean, well-scoped, and dependable

Jakub Skibicki 10/10

Kuba built the foundation - the function definitions and their chosen domains (`amr/functions.py`), input sampling (`amr/data.py`), the mutant operators (`amr/mutants.py`), and the metrics (`amr/metrics.py`). The domain choices look simple but they are load-bearing. Capping tan to $[-1.5, 1.5]$ to dodge the asymptotes and clamping the log functions to their valid range is what keeps the search from blowing up, and getting that wrong would have wasted everyone's time downstream. The mutant operators and the kill-rate metric define how we score ourselves. He also set up the project and wrote tests across all four of his modules